

Basic Quantitative Method: TA Session Week 2

Danny Po-Hsien Kang (康柏賢)

July 10, 2026

Today's agenda

1 Data Visualization

- `geom_histogram()`
- `geom_bar()`
- `labs()`
- `geom_point()`
- `scale_x_continuous()`, `scale_y_continuous()`
- `geom_boxplot()`
- `ggsave()`

Today's Dataset

- Today we will use the following csv files from the OpenIntro Website:
 - `loan_full_schema.csv`: The dataset represents thousands of loans made through the Lending Club platform.
 - `ames.csv`: The dataset includes the housing prices in Ames, Iowa, from 2006 to 2010.
 - `ncbirths.csv`: A 2004 dataset from North Carolina on births, including mothers' habits and practices, with 1,000 cases.
- Please import the following datasets into RStudio.

```
loan <- read.csv("data/loans_full_schema.csv")  
housing <- read.csv("data/ames.csv")  
birth <- read.csv("data/ncbirths.csv")
```

Histogram: `geom_histogram()`

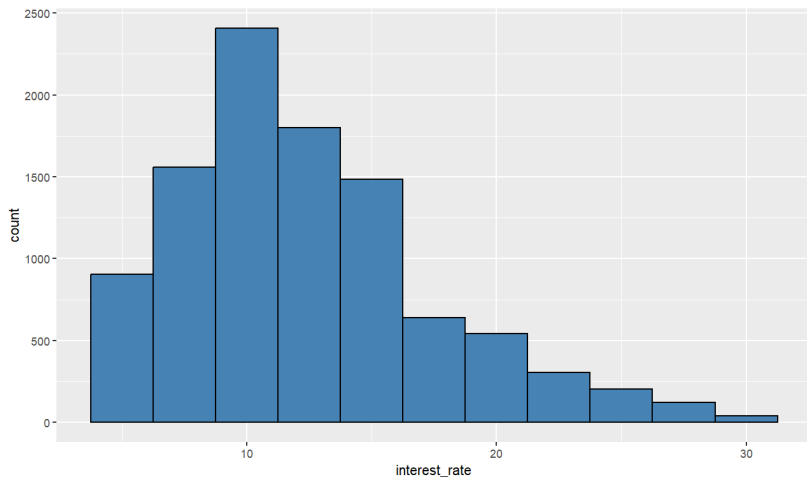


Figure: Histogram of interest rates.

Histogram: geom_histogram()

- **Syntax:**

```
data %>%  
  ggplot(aes(x = ...)) +  
  geom_histogram(binwidth = ...,  
                 fill = ...,  
                 color = ...  
                )
```

- **Example:**

```
loan %>%  
  ggplot(aes(x = interest_rate)) +  
  geom_histogram(binwidth = 2.5,  
                 fill = "steelblue",  
                 color = "black"  
                )
```

Barplot: `geom_bar()`

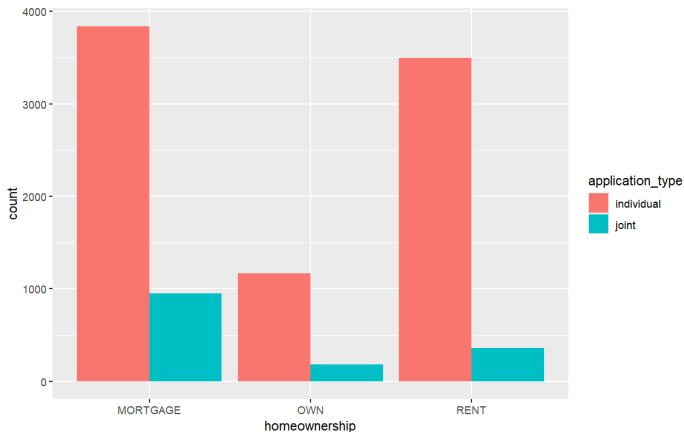


Figure: Bar plot of homeownership by application type.

Barplot: geom_bar()

- **Syntax:**

```
loan %>%  
  ggplot(aes(x = ..., fill = ...)) +  
  geom_bar(position = ...)
```

- **Example:**

```
loan %>%  
  ggplot(aes(x = homeownership, fill = application_type)) +  
  geom_bar(position = "dodge") # stack, fill, ...
```

Add Title and Axis Labels: labs()

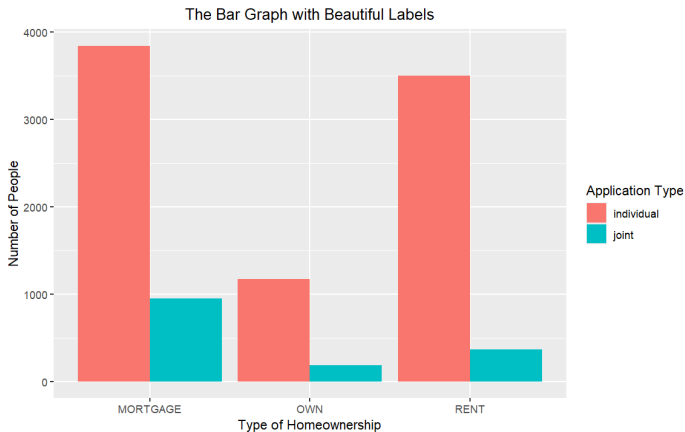


Figure: Same bar plot, **but with title and labels.**

Add Title and Axis Labels: labs()

- **Syntax:**

```
Your_Plot +  
  labs(x = ..., y = ..., title = ...)
```

- **Example:**

```
loan %>%  
  ggplot(aes(x = homeownership, fill = application_type)) +  
  geom_bar(position = "dodge") +  
  labs(  
    x = "Type of Homeownership",  
    y = "Number of People",  
    title = "The Bar Graph with Beautiful Labels",  
    fill = "Application Type"  
  )
```

Scatter Plot: `geom_point()`

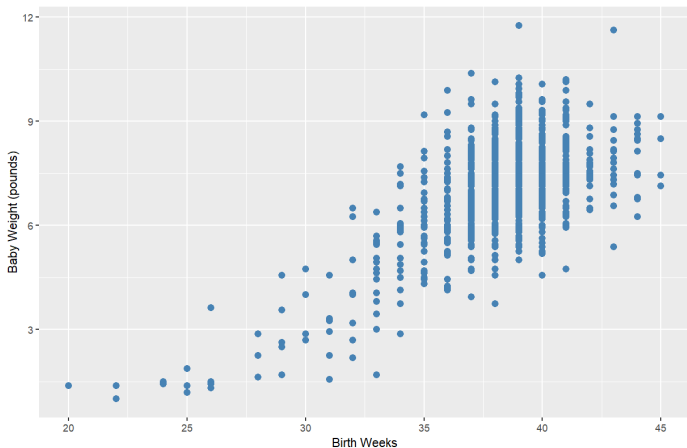


Figure: Scatter plot of multiple unit ratio and home ownerships.

Scatter Plot: `geom_point()`

- **Syntax:**¹

```
ggplot(data, aes(x = ..., y = ...)) +  
  geom_point(color = ...,  
             size = ...,  
             shape = ...  
            )
```

- **Example:**²³

```
ggplot(birth, aes(x = weeks, y = weight)) +  
  geom_point(color = "steelblue", size = 2.5) +  
  labs(  
    x = "Birth Weeks",  
    y = "Baby Weight (pounds)"  
  )
```

¹There are more options in the function, see [here](#).

²R color [cheatsheet](#).

³ggplot2 [Quick Reference](#): shape

Adjust Scale of x, y Axes: `scale_x_continuous()`

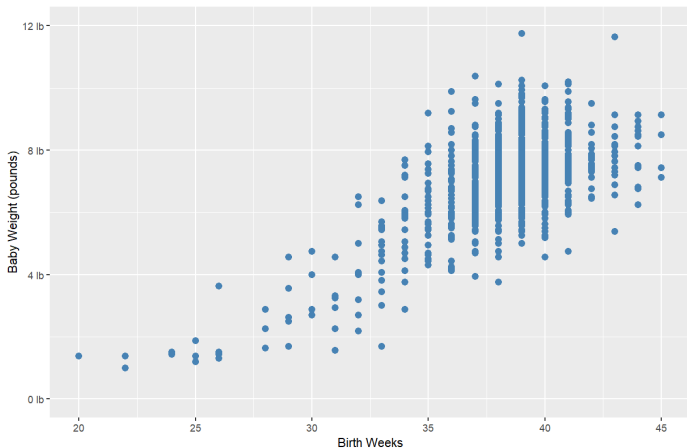


Figure: Scatter plot with adjusted scale.

Adjust Scale of x, y Axes: scale_x_continuous()

• Syntax:

```
Your_Plot +  
  scale_x_continuous(breaks = ..., labels = ..., limits = ...)  
  scale_y_continuous(breaks = ..., labels = ..., limits = ...)
```

• Example:

```
.. +  
  scale_x_continuous(  
    limits = c(20, 45),  
    breaks = c(20, 25, 30, 35, 40, 45)  
  ) +  
  scale_y_continuous(  
    limits = c(0, 12),  
    breaks = c(0, 4, 8, 12),  
    labels = c("0 lb", "4 lb", "8 lb", "12 lb")  
  )
```

Boxplot: geom_boxplot()

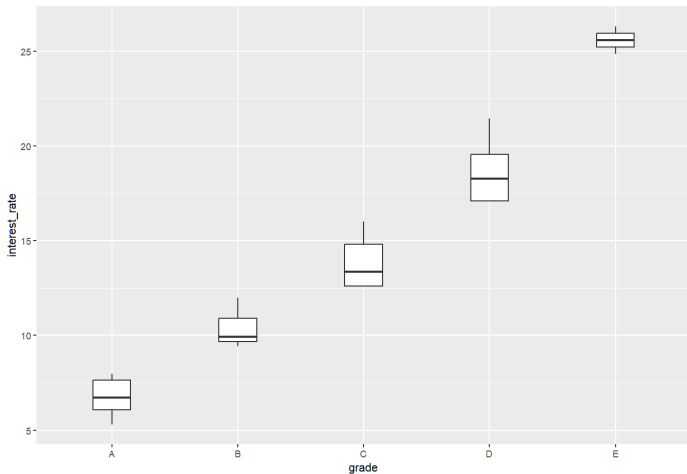


Figure: Box plot of interest rates by verified income.

Boxplot: geom_boxplot()

- **Syntax:**

```
ggplot(data, aes(x = ..., y = ...)) +  
  geom_boxplot(width = ...) #optional
```

- **Example:**

```
ggplot(loan, aes(x = grade, y = interest_rate)) +  
  geom_boxplot(width = 0.3)
```

Save Your Graph into File: ggsave()

- **Syntax:**

```
ggsave(filename = ...,  
        plot = ...,  
        path = ...,  
        width = ...,  
        height = ...,  
        dpi = ...)
```

- **Example:**

```
box_plot <- ggplot(loan, aes(x = grade, y = interest_rate)) +  
  geom_boxplot(width = 0.3)
```


ggsave(): Which plot is saved?

- By default, ggsave() saves the newest plot showed in the plot window.

```
print(plot_1)
print(plot_2)
ggsave(filename = ...) #It saves plot_2!
```

- Use `plot = ...` to assign which plot you want to save. (Or just put `ggsave()` right after your plot produced.)
- Use `filename = ...` to set the file names and the output type (`.png`, `.jpeg` ...).
- Example:**

```
ggsave(filename = "box_plot.png",
        plot = box_plot)
```

ggsave(): Where is my plot saved?

- By default, ggsave() saves your plot to the working directory.
 - It saves to the project folder if you have opened one.
 - But it is better to save in the plots folder!
- Two ways to assign the saved path:
 - Type the path inside filename = ... (absolute or relative).
 - Or use path = ...
- **Example:**

```
ggsave(filename = "C:/Users/you/figures/box_plot.png")  
ggsave(filename = "figures/box_plot.png")  
ggsave(filename = "box_plot.png", path = "figures")
```

ggsave(): How big is my plot?

- By default, ggsave() uses the current plot size in the RStudio Plots window.
 - Which is not good because you may produce difference size of plots every time!
- Use `width = ...` and `height = ...` to control the size.
- Use `dpi = ...` to control the quality of the plot (default = 300).
- **Example:**

```
ggsave(filename = "box_plot.png",  
        width = 6,  
        height = 4,  
        dpi = 300)
```

ggsave(): Put everything together

- **Example:**

```
ggsave(filename = "box_plot.png",  
        plot = box_plot,  
        path = "figures",  
        width = 6,  
        height = 4,  
        dpi = 300)
```